

TENTATIVE REPORT

LiDAR Based SLAM Using ICP (Iterative Closest Point) Algorithm

PDMG- Payload Data Management Group

Batch - 60

18-03-2025 to 04-07-2025

Team Members

R. Bhanu Prasad ,

Harishwar J,

P. Yashashwini,

R Rajvelu,

Project Guide

Surabhi Bothra
Scientist/Engineer - 'SE'
DSD, Payload Data Management Group (PDMG)

1 SLAM Overview

1.1 INTRODUCTION

Simultaneous Localization and Mapping (SLAM) is a foundational concept in robotics and autonomous systems. It is the computational challenge of constructing a map of an unknown environment while simultaneously keeping track of an agent's location within that map. The SLAM problem is fundamentally probabilistic. A robot's sensors and its own movements have inherent uncertainty. Therefore, SLAM algorithms must manage these uncertainties to produce a consistent map and a reliable estimate of the robot's trajectory. As a robot moves through an environment, it continually gathers new sensor data and updates its estimates for both the map and its own position. A critical element in many SLAM systems is **loop closure**, which is the ability to recognize a previously visited location. Correctly identifying a loop closure allows the system to significantly reduce accumulated errors and improve the overall consistency of the map.

For a robot or any autonomous agent to navigate and interact with its surroundings intelligently, it must answer two fundamental questions: "Where am I?" (Localization) and "What does the world around me look like?" (Mapping). The challenge lies in the fact that these two problems are interdependent; a good map is necessary for accurate localization, and accurate localization is required to build a good map. This cyclical dependency is the core problem that SLAM algorithms aim to solve.

1.2 SLAM PROBLEM DEFINITION

The SLAM problem involves a robot simultaneously estimating its location and building a map of an unknown area. Since sensors have inherent noise, this can't be done perfectly. Instead, SLAM is framed as a probability problem. The goal is to determine the most likely robot **trajectory** ($x_{1:T}$) and **map** (m) based on sensor data. This is calculated from the robot's **odometry measurements** ($u_{1:T}$), its environmental **sensor readings** ($z_{1:T}$), and its initial starting **pose** (x_0).

Mathematically, the problem is defined as finding the posterior probability:

$$p(x_{1:T}, m \mid z_{1:T}, u_{1:T}, x_0)$$

Here, the initial pose, x_0, typically serves as the origin for the global map's coordinate system. In a 2D system, odometry is often represented by the robot's change in position and orientation (x,y,θ).

1.3 Key Aspects of the SLAM Problem

At its core, SLAM is a "chicken-and-egg" problem because a robot needs a good map to determine its location, but it also needs to know its location to build a good map. This interdependence is the central difficulty that SLAM algorithms are designed to overcome.

The process involves two main, ongoing tasks:

- **Mapping:** This is the process of creating a model or representation of the surrounding environment. This map can take various forms, such as a grid of occupied spaces, a set of geometric features, or a dense 3D point cloud.
- **Localization:** This is the task of estimating the robot's position and orientation (its "pose") within the map it is building.

The applications of SLAM are vast and transformative, underpinning technologies such as:

- **Autonomous Vehicles:** For navigation, obstacle avoidance, and path planning.
- **Robotics:** Enabling indoor robots for tasks like automated vacuuming, warehouse management, and exploration.
- **Augmented & Virtual Reality (AR/VR):** For tracking the user's position and overlaying digital information onto the real world.
- **Unmanned Aerial Vehicles (UAVs/Drones):** For aerial surveying, inspection, and navigating GPS-denied environments.

1.4 TYPES OF SLAM

SLAM is not a single algorithm but a family of them. The two primary approaches are **filtering-based SLAM** and **optimization-based (or graph-based) SLAM**.

1.4.1 FILTERING-BASED SLAM

This was the earliest approach to solving the SLAM problem. It treats SLAM as a state estimation problem, where the state includes the robot's pose and the map features. A filter, like an **Extended Kalman Filter (EKF)** or a **Particle Filter**, is used to recursively estimate the posterior probability of the state as new measurements arrive.

- **EKF SLAM:** Was one of the first successful SLAM solutions but struggles with large-scale environments due to its computational complexity, which grows quadratically with the number of map features.
- **Particle Filter SLAM (e.g., GMapping):** Uses a set of "particles" to represent different hypotheses of the robot's pose. It is more robust than EKF for non-linear problems but can be computationally intensive.

1.4.2 GRAPH-BASED SLAM (POSE GRAPH OPTIMIZATION)

This is the modern, state-of-the-art approach and is more scalable for large environments. It tackles the problem in two main stages: the front-end and the back-end.

1. **Front-End (Scan Matching):** The front-end is responsible for generating an initial estimate of the robot's motion. It does this by taking the current sensor reading (e.g., a LiDAR scan) and aligning it with the previous

one. This process, known as **scan matching** or registration, provides an estimate of the robot's relative movement (odometry). Common scan matching algorithms include:

- **Iterative Closest Point (ICP):** An algorithm that iteratively refines the transformation (rotation and translation) needed to align two point clouds.
- **Normal Distributions Transform (NDT):** A technique that models the environment as a set of normal distributions, which can be more efficient than ICP in some cases.

2. **Back-End (Graph Optimization):** The front-end odometry inevitably accumulates drift over time. The back-end's job is to correct this. It builds a **pose graph**, where each node represents a robot pose (a position in time) and the edges represent the spatial constraints between them (the relative motion from the front-end). When the robot recognizes a place it has been before (**loop closure**), a new edge is added to the graph. The back-end then optimizes this entire graph to find the most probable configuration of poses, effectively minimizing the drift and producing a globally consistent map.

2 Methodology of LiDAR SLAM

2.1 Fundamental Approaches to Position Estimation

In SLAM, the primary goal is to estimate the robot's position (and orientation), also known as its "pose." Methodologies for estimating this pose are generally divided into two main categories:

1. **Filtering-based Methods:** This is the classical approach where the SLAM problem is treated as a state estimation problem. A filter, such as an **Extended Kalman Filter (EKF)** or a **Particle Filter**, recursively estimates the robot's pose and the map as new sensor data becomes available. This method processes data sequentially and does not typically re-evaluate past poses, which can lead to the accumulation of errors.
2. **Optimization-based (Graph-based) Methods:** This is the modern, more scalable approach. It treats SLAM as a graph optimization problem. The core idea is to build a **pose graph** where each **node** represents a robot pose at a specific time, and the **edges** between nodes represent the spatial constraints between them (i.e., the robot's movement from one pose to the next). This approach consists of two main parts:

    i. **Front-End:** Responsible for generating the initial estimates of motion by matching consecutive sensor scans. The ICP algorithm used in the provided notebook is a perfect example of a SLAM front-end. It provides the odometry estimates that form the initial edges of the graph.

    ii. **Back-End:** Responsible for optimizing the entire pose graph. When the robot recognizes a previously visited location (a "loop closure"), a new constraint is added to the graph. The back-end optimizer then adjusts all the poses in the graph to minimize the overall error, correcting the drift that has accumulated over time.

The ICP-based approach in the notebook is a front-end technique that falls under the umbrella of optimization-based SLAM. It provides the odometry but does not perform the back-end graph optimization or loop closure detection.

2.2 Environmental Representation in SLAM

A map in SLAM is a representation of the environment. The choice of representation depends on the sensor and the application. Common types include:

- **Occupancy Grids:** The environment is divided into a grid of cells, with each cell storing the probability of it being occupied or free. This is common for 2D SLAM.
- **Feature Maps:** The map consists of a set of distinct landmarks or features (e.g., corners, doors, or specific visual features). This was common in early SLAM systems.
- **Point Clouds:** This is the most direct representation for LiDAR data. The map is simply a large collection of 3D points that represent the surfaces of the environment.

In the provided notebook, the map representation is a **point cloud** (*map_accumulator* in the code). This map is built by taking the raw, full-resolution LiDAR scan at each step, transforming it using the estimated global pose, and merging it with the existing map. This process creates a dense, 3D model of the environment directly from the sensor data.

2.3 The Core of Odometry: Scan Matching with ICP

Registration, also known as scan matching, is the process of aligning two or more sets of sensor data into a single, consistent coordinate system. In the context of the notebook, it is the core process used to determine the robot's movement between two consecutive time steps. The notebook uses the **Iterative Closest Point (ICP)** algorithm for this purpose.

ICP is an algorithm that finds the optimal rotation and translation to align two point clouds. It works as follows:

1. **Find Correspondences:** For each point in the source point cloud, it finds the closest corresponding point in the target point cloud.
2. **Estimate Transformation:** It calculates the rotation and translation that best aligns these corresponding pairs, typically by minimizing the mean squared error between them.
3. **Apply Transformation:** It applies this transformation to the source point cloud.
4. **Iterate:** Steps 1-3 are repeated until the alignment converges (i.e., the change in alignment error falls below a threshold or a maximum number of iterations is reached).

The notebook employs a more robust variant called **point-to-plane ICP**. Instead of minimizing the distance between points, it minimizes the distance between points in the source cloud and the plane defined by the corresponding point and its normal in the target cloud. This is why the code explicitly estimates normals for the point clouds before registration and uses `o3d.pipelines.registration.TransformationEstimationPointToPlane()`. This method is generally more accurate, especially in structured environments with flat surfaces.

2.4 Integrating ICP into a Graph-Based Framework

While the notebook focuses on ICP for odometry, this is the foundational "front-end" for a full graph-based SLAM system. Here's how it connects:

1. Graph Construction (The Front-End):

- The **poses** calculated at each time step by the ICP algorithm would become the **nodes** in a pose graph.
- The **relative transformation** between consecutive poses (`current_relative_transform` in the code) would become the **edge** connecting those nodes. This edge represents the odometric constraint.

2. Drift Accumulation:

- As seen in the notebook's evaluation, this sequential process accumulates small errors over time, leading to a significant **Absolute Trajectory Error (ATE)** of 12.2175 meters. The estimated trajectory slowly drifts away from the true path.

3. Graph Optimization (The Back-End):

- A full graph-based SLAM system would add a **back-end** to correct this drift. The back-end's primary job is to perform **loop closure detection**. This involves recognizing when the robot has returned to a previously visited area.
- When a loop closure is detected, a new edge is added to the graph between the current pose and the previously visited, non-sequential pose. This new edge acts as a powerful constraint.
- An optimizer then adjusts all the nodes (poses) in the graph to satisfy all the constraints simultaneously, minimizing the overall error. This process effectively "pulls" the drifted parts of the trajectory back into place, resulting in a globally consistent map and a much more accurate trajectory.

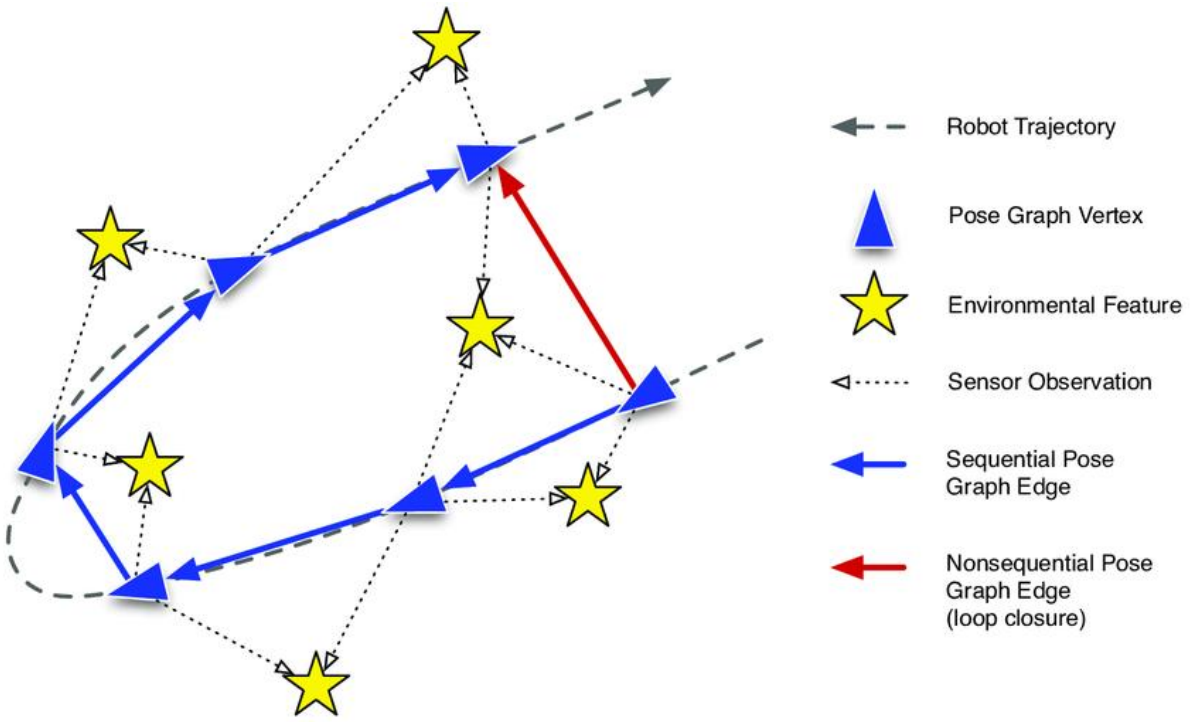


Fig 1: *A schematic depiction of a pose graph. The vertices in the graph correspond to poses where sensor observations were made. Matching sensor observations give rise to constraints on the edges. Sequential edges follow the trajectory, non-sequential edges "close loops".*

In summary, the ICP registration performed in the notebook generates the essential inputs for a graph-based SLAM system. To create a complete solution, one would need to add a graph optimization back-end with loop closure capabilities to correct the inherent drift of the ICP-based odometry.

3 Data Acquisition and Dataset Selection

The foundation of any SLAM system evaluation is a robust and appropriate dataset. The dataset must not only provide clean sensor data but also represent a challenging and realistic environment for the algorithm. Our data acquisition process involved two distinct phases: an initial exploration with an indoor dataset and a final implementation using a more extensive outdoor dataset.

3.1 Initial Dataset Exploration: The MI3DMap Dataset

Our initial efforts utilized the **MI3DMap dataset**, specifically the "mimap in slam 00" sequence. This dataset features:

- **Environment:** A two-floor indoor building scene.
- **Sensor Data:** Scans were captured using a **Velodyne Ultra Puck** LiDAR. The environment included individual rooms, non-enclosed loop corridors, and stairs.
- **Ground Truth:** A ground truth point cloud was provided, which was generated using a high-precision Riegl VZ-1000 scanner, covering the corridors and stairs.

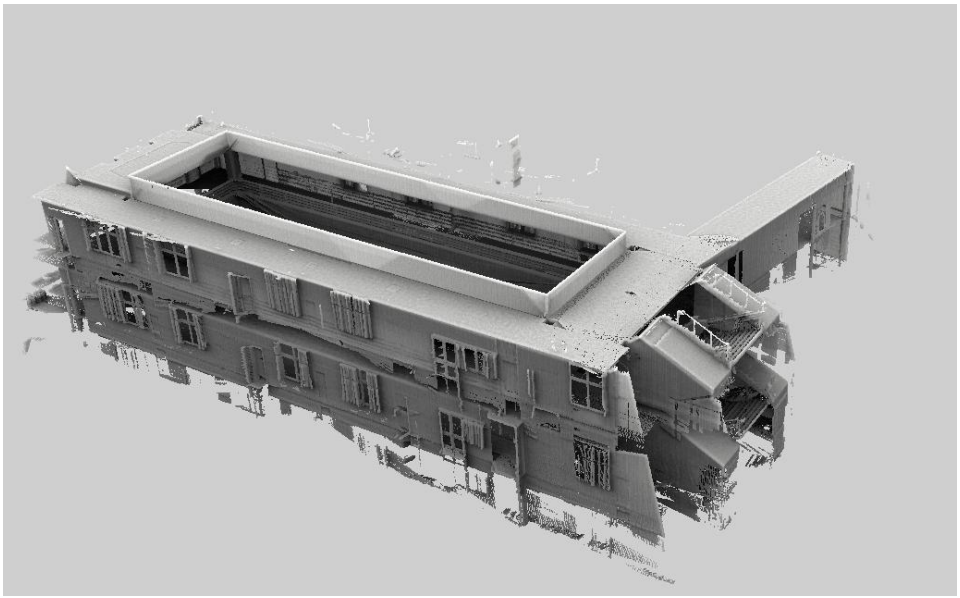


Fig 3: A two-floor building scene. The scans scanned by a Velodyne Ultra pack include data of individual rooms, non-enclosed loop corridors and stairs. Ground truth point cloud scanned by a Rigel VZ 1000 includes the corridors and stairs.

While this dataset provided a structured indoor environment, the results generated by our ICP-based odometry were not satisfactory for our project goals. The characteristics of the indoor space and the nature of the sensor data did not align well with the intended application, leading us to seek an alternative.

3.2 Final Dataset Selection: The RVP SLAM "Campus" Dataset

After evaluating the initial results, we selected the **RVP SLAM "Campus" dataset** for our final implementation and testing. This dataset was more aligned with testing a LiDAR odometry system in a large-scale, dynamic environment. The specific sequence used was **"Train0."**

The key features of the "Campus" dataset are:

- **Environment:** An expansive outdoor urban setting, captured at the main campus of Sapienza University. The environment includes multiple streets, several narrow passages, and tunnels under buildings, presenting a diverse set of challenges for a SLAM algorithm.
- **Motion and Dynamics:** The data was acquired using a car, making it ideal for autonomous vehicle applications. The environment has low dynamics, primarily consisting of people walking.
- **Scale and Richness:** The "Train0" sequence has a duration of **2290 seconds** and covers a total length of **11.455 km**, featuring numerous loops. The presence of loops is particularly valuable for evaluating the long-term drift of an odometry system and provides the necessary features for potential loop closure and back-end optimization.
- **LiDAR Specifications:** The data was collected with a high-resolution LiDAR sensor operating at **20 Hz** with a **360° x 45°** field of view.

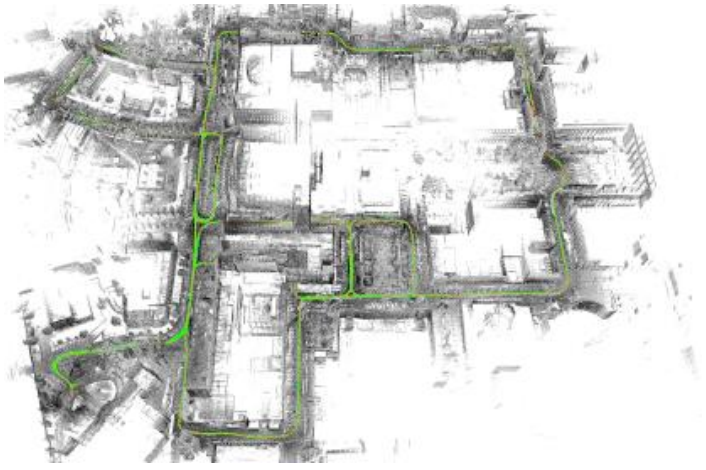


Fig 3: The main Campus of Sapienza University

This dataset proved to be far more suitable, providing a challenging, large-scale environment with the necessary ground truth data to thoroughly evaluate the performance of our ICP-based odometry implementation. The code provided in the notebook was specifically adapted and run using this dataset.

4 Data Preparation and Pre-processing

Before the SLAM algorithm can be implemented, the raw sensor data, stored in a ROS (Robot Operating System) bag file, must be inspected and converted into a format suitable for processing. This chapter details the critical pre-processing steps undertaken to prepare the "Campus" dataset for our ICP-based odometry pipeline.

4.1 Inspecting the Bag File Contents

The first step in working with any ROS dataset is to understand its structure and contents. ROS bag files are archives that can contain messages from multiple sensor topics. We used the standard ROS command-line tool, `rosbag`, to inspect the `campus_train0_01.bag` file.

The ‘ `rosbag info`’ command provides a summary of the bag file, including its duration, size, and a list of all the topics it contains, along with their message counts and types.

Command Executed: “ `rosbag info campus_train0_01.bag` ”

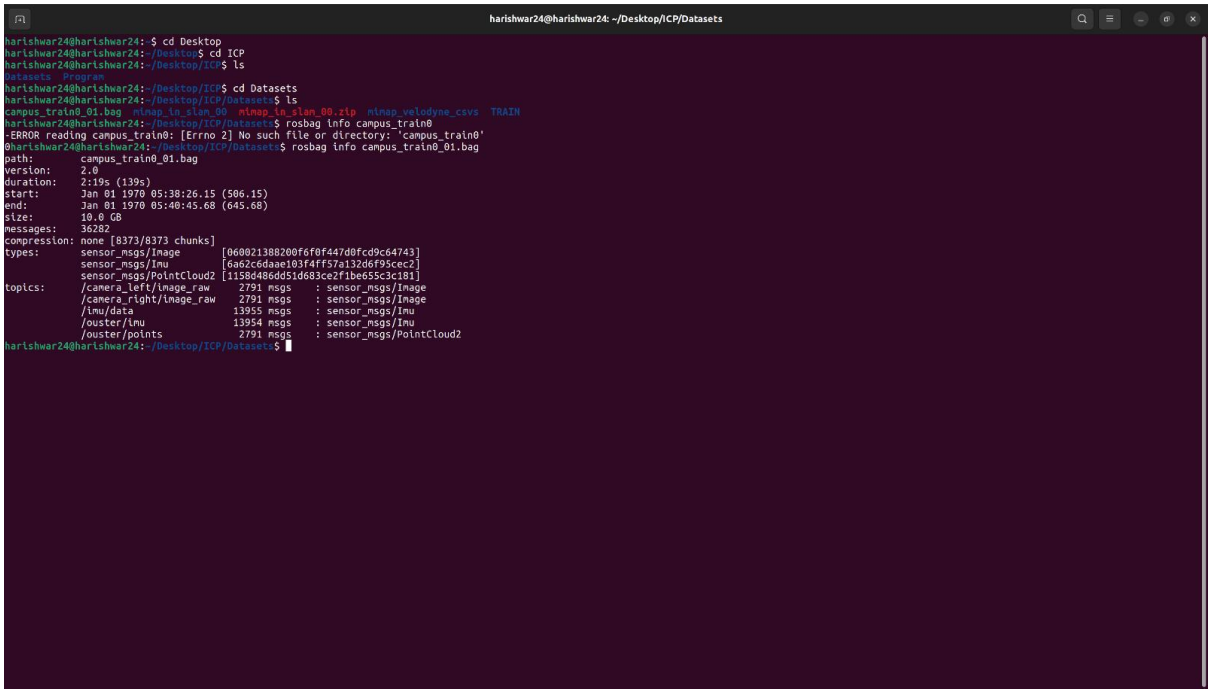


Fig 4: Inspecting the Bag File Contents by using `rosbag info` *command line*

Output and Analysis:

The output from this command revealed several sensor streams, as shown in the user-provided screenshot. The key topics identified were:

- `/camera_left/image_raw` & `/camera_right/image_raw`: Stereo camera feeds.
- `/imu/data` & `/ouster/imu`: Data from two separate Inertial Measurement Units (IMUs).
- `/ouster/points`: The primary 3D point cloud data from the Ouster LiDAR sensor.

For our LiDAR-based SLAM implementation, the `/ouster/points` topic is the one of interest. The output confirmed it contains **2793 messages** of the type `sensor_msgs/PointCloud2`, which is the standard format for point cloud data in ROS. This confirmed we had the correct topic name and a substantial number of scans to process.

4.2 Extracting LiDAR Scans to Timestamped CSV Files

While ROS bags are efficient for recording data, the `sensor_msgs/PointCloud2` message format is a binary data blob that is not directly usable by many data analysis tools and Python libraries like `open3d` and `pandas`. Therefore, the next crucial step was to extract each point cloud scan from the bag file and save it as a separate, human-readable CSV file.

To accomplish this, the following Python script was developed.

4.2.1 Data Extraction Script: (ROS Bag to CSV Converter)

```
import rosbag
import sensor_msgs.point_cloud2 as pc2
import numpy as np
import pandas as pd
import os

def convert_ros1bag_to_timestamped_csvs(bag_file_path, lidar_topic, output_dir,
                                        fields_to_extract=['x', 'y', 'z', 'intensity'],
                                        filename_prefix='scan_'):
    """
    Reads PointCloud2 messages from a ROS 1 bag file, extracts specified fields,
    and saves each message to a separate CSV file named with its timestamp.
    """

    if not os.path.exists(output_dir):
        os.makedirs(output_dir)
        print(f"Created output directory: {output_dir}")

    print(f"Opening ROS 1 bag file: {bag_file_path}")

    try:
        bag = rosbag.Bag(bag_file_path, 'r')
    except Exception as e:
        print(f"Error opening bag file: {e}")
        return

    total_messages_on_topic = bag.get_message_count(topic_filters=[lidar_topic])
    if total_messages_on_topic == 0:
        print(f"Error: No messages found on the specified topic '{lidar_topic}'. Please check the topic name.")
```

```
            bag.close()
            return

        print(f"Reading messages from topic: '{lidar_topic}' ({total_messages_on_topic} total messages)")
        print(f"Extracting fields: {fields_to_extract}")

        message_count = 0
        for topic, msg, t in bag.read_messages(topics=[lidar_topic]):
            message_count += 1
            print(f"Processing message {message_count}/{total_messages_on_topic}...")

            try:
                # Get the timestamp (seconds.nanoseconds as a float)
                timestamp_sec = msg.header.stamp.to_sec()

                # Define the output CSV filename
                csv_filename = f"{filename_prefix}{timestamp_sec:.9f}.csv"
                csv_filepath = os.path.join(output_dir, csv_filename)

                # Use the point_cloud2 library to iterate through the points
                points_generator = pc2.read_points(msg, field_names=fields_to_extract, skip_nans=True)

                # Convert generator to a list of lists
                points_list = [list(p) for p in points_generator]

                if not points_list:
                    print(f"  Warning: Message at timestamp {timestamp_sec:.9f} had no valid points. Skipping.")
                    continue

                # Create a pandas DataFrame
                df = pd.DataFrame(points_list, columns=fields_to_extract)
                # Save the DataFrame to a CSV file
                df.to_csv(csv_filepath, index=False)

            except Exception as e:
                print(f"  Error processing message at ROS time {t.to_sec()}: {e}")
                continue

        bag.close()
        print(f"\nProcessing complete.")
        print(f"Total CSV files created: {message_count} in '{output_dir}'")

    if __name__ == '__main__':

        BAG_FILE = '/home/harishwar24/Desktop/ICP/Datasets/campus_train0_01.bag'

        LIDAR_TOPIC = '/ouster/points'

        OUTPUT_CSV_DIR = 'train01_timestamped_csv'

        FIELDS = ['x', 'y', 'z', 'intensity', 'ring']

        print("--- Starting ROS 1 Bag to CSV Conversion ---")
        convert_ros1bag_to_timestamped_csvs(
```

```
    bag_file_path=BAG_FILE,
    lidar_topic=LIDAR_TOPIC,
    output_dir=OUTPUT_CSV_DIR,
    fields_to_extract=FIELDS
)
print("--- Conversion Script Finished ---")
```

4.2.2 Code Explanation:

The script systematically processes the `.bag` file to produce the required CSV data.

1. **Function Definition:** The core logic is within the `convert_ros1bag_to_timestamped_csvs` function. It takes the bag file path, the target LiDAR topic, an output directory, and the desired fields as input.
2. **File Handling:** It opens the ROS bag file for reading and verifies that the specified `lidar_topic` exists and contains messages.
3. **Message Iteration:** The script reads through the bag, processing only the messages from the `/ouster/points` topic.
4. **Timestamping:** For each message, it extracts the timestamp from the message header. This is critical as it provides the precise time each scan was captured. The timestamp is used to generate a unique filename for each CSV (e.g., `scan_1571160010.123456789.csv`), preserving the temporal order of the data.
5. **Data Parsing:** The `sensor_msgs.point_cloud2.read_points` function is used to parse the raw binary point cloud data. It iterates through each point in the scan and extracts the specified fields: `x`, `y`, `z` (coordinates), `intensity` (reflectivity), and `ring` (the laser ring that generated the point).
6. **CSV Conversion:** The list of extracted points is then loaded into a `pandas` DataFrame, which provides a convenient and efficient way to structure the data. This DataFrame is then written to its corresponding timestamped CSV file.

Upon completion, this script produces a directory containing 2793 individual CSV files, each representing a single LiDAR scan. This organized and accessible dataset is now ready to be loaded and processed by the main ICP SLAM algorithm, as detailed in the subsequent chapters.

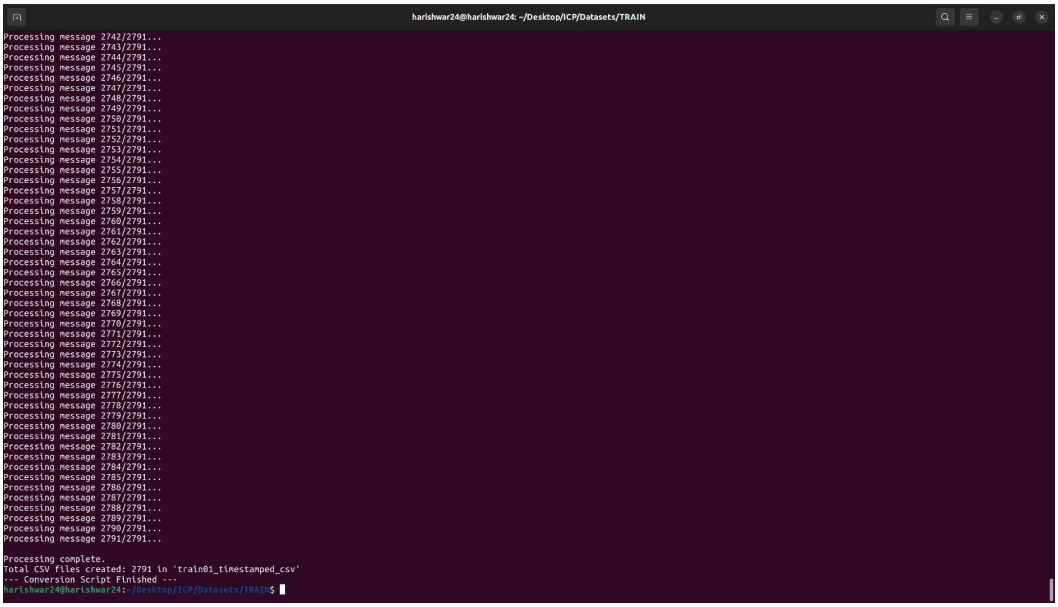


Fig 5: extracting the lidar scan values from the bag file and saving it with timestamped csv values for all the available lidar cloud points scans in the bag files csv.

5 Algorithm Implementation and Core Functions

This chapter details the key functions and the main algorithmic loop used to implement the Iterative Closest Point (ICP) based SLAM front-end. The implementation relies heavily on the open3d library for point cloud manipulation and registration, and numpy for handling transformations.

The overall process can be broken down into three main stages: loading and preprocessing the data, executing the iterative ICP odometry loop, and finally, saving the results.

Stage 1: Data Loading and Preprocessing

This initial phase is responsible for taking the raw input data (the timestamped CSV files) and converting it into a format suitable for the main SLAM algorithm. This involves loading each CSV into an `open3d.PointCloud` object and then preprocessing these point clouds to make the ICP registration more efficient and robust.

Functions:

```
import open3d as o3d
import numpy as np
import pandas as pd
import os
import glob


def load_csv_as_point_cloud(csv_filepath):
    try:
        # Load CSV using pandas for efficiency
        df = pd.read_csv(csv_filepath)
```

```
        # Ensure the required columns exist
        if not all(col in df.columns for col in ['x', 'y', 'z']):
            return None

        pcd = o3d.geometry.PointCloud()
        # Convert the pandas DataFrame to a numpy array and then to a Vector3dVector
        pcd.points = o3d.utility.Vector3dVector(df[['x', 'y', 'z']].values)
        return pcd
    except Exception as e:
        print(f"Error loading CSV {os.path.basename(csv_filepath)}: {e}")
        return None

def preprocess_scans(all_scans_original, voxel_downsample_size):
    all_scans_downsampled = []
    print(f"Downsampling {len(all_scans_original)} scans and estimating normals...")
    for scan in all_scans_original:
        down_pcd = o3d.geometry.PointCloud()
        if scan.has_points():
            # Voxel downsampling reduces the number of points for faster ICP registration
            down_pcd = scan.voxel_down_sample(voxel_downsample_size)
            # Normals are essential for the more robust point-to-plane ICP variant
            down_pcd.estimate_normals(
                search_param=o3d.geometry.KDTreeSearchParamHybrid(radius=0.5, max_nn=30)
            )
        all_scans_downsampled.append(down_pcd)
    print("Downsampling and normal estimation complete.")
    return all_scans_downsampled
```

Explanation:

- `load_csv_as_point_cloud`: This function takes the file path of a single CSV scan, reads it using `pandas`, and converts the `x, y, z` columns into an `open3d.geometry.PointCloud` object.
- `preprocess_scans`: This is a critical step for efficiency. It takes the list of all original, high-resolution scans and performs two main operations:

1. **Voxel Downsampling:** It reduces the number of points in each scan. This significantly speeds up the ICP registration process, which is computationally expensive.
2. **Normal Estimation:** It calculates the normal vector for each point. These normals are essential for using the more accurate and robust **point-to-plane ICP** variant, as opposed to the basic point-to-point method.

Stage 2: Core SLAM Algorithm: ICP Odometry Loop

This is the main algorithmic block where the robot's trajectory is estimated. The `run_icp_odometry` function initializes the SLAM state and then enters a loop. In each iteration, it registers the current LiDAR scan to the previous one to find the relative motion. This motion is then used to update the robot's global pose and build up a composite map of the environment.

Function:

```
import open3d as o3d
import numpy as np

# --- Configuration Parameters (defined globally for the algorithm) ---
VOXEL_DOWNSAMPLE_SIZE = 0.2
MAP_VOXEL_DOWNSAMPLE_SIZE = 0.4
ICP_MAX_CORRESPONDENCE_DISTANCE = 0.8
ICP_MAX_ITERATIONS = 50
ICP_FITNESS_THRESHOLD = 0.6 # A score from 0 to 1 indicating the quality of the alignment

def run_icp_odometry(all_scans_original, all_scans_downsampled, all_timestamps):
    # --- Initialization ---
    # The list of global poses starts with an identity matrix (origin)
    global_poses = [np.identity(4)]
    # The map accumulator is initialized with the first scan
    map_accumulator = o3d.geometry.PointCloud(all_scans_original[0])
    map_accumulator.transform(global_poses[0])

    # A motion prior for ICP, initialized as identity. It's updated with the last successful transform.
    previous_relative_transform = np.identity(4)
    # Point-to-plane is generally more robust than point-to-point ICP
    estimation_method = o3d.pipelines.registration.TransformationEstimationPointToPlane()

    print(f"Processing {len(all_scans_downsampled) - 1} scan pairs...")
```

```
# --- Main Loop ---
for i in range(len(all_scans_downsampled) - 1):
    # The previous scan is the target
    target_scan_down = all_scans_downsampled[i]
    # The current scan is the source, which we want to align to the target
    source_scan_down = all_scans_downsampled[i+1]

    # Use the transformation from the previous step as an initial guess.
    # This assumes relatively constant velocity and helps ICP converge faster.
    trans_init = previous_relative_transform

    # --- ICP Registration ---
    reg_icp = o3d.pipelines.registration.registration_icp(
        source=source_scan_down,
        target=target_scan_down,
        max_correspondence_distance=ICP_MAX_CORRESPONDENCE_DISTANCE,
        init=trans_init,
        estimation_method=estimation_method,
        criteria=o3d.pipelines.registration.ICPConvergenceCriteria(
            max_iteration=ICP_MAX_ITERATIONS
        )
    )

    # The result is the relative transformation from the source to the target frame
    current_relative_transform = reg_icp.transformation

    # Check registration quality. If fitness is high, the alignment is good.
    if reg_icp.fitness > ICP_FITNESS_THRESHOLD:
        # Update the motion prior only on successful registration
        previous_relative_transform = current_relative_transform
    else:
```

```
        # If fitness is too low, the result is unreliable. Assume no motion.
        current_relative_transform = np.identity(4)

    # --- Pose and Map Update ---
    # Update the global pose by composing the last global pose with the new relative transform
    current_global_pose = global_poses[-1] @ current_relative_transform
    global_poses.append(current_global_pose)

    # Add the current (full-resolution) scan to the map accumulator,
    # transformed by its new global pose.
    source_copy = o3d.geometry.PointCloud(all_scans_original[i+1])
    source_copy.transform(current_global_pose)
    map_accumulator += source_copy

    # Periodically downsample the map to keep it from becoming too large
    if (i + 1) % 30 == 0:
        map_accumulator = map_accumulator.voxel_down_sample(MAP_VOXEL_DOWNSAMPLE_SIZE)

    if (i + 1) % 25 == 0:
        print(f"Processed scan {i+1}. Fitness: {reg_icp.fitness:.4f}, RMSE: {reg_icp.inlier_rmse:.4f}")

return global_poses, map_accumulator
```

Explanation:

- **Initialization:** The `global_poses` list is created to store the 4x4 transformation matrix for each step, starting with the identity matrix at the origin. The `map_accumulator` is initialized with the very first scan.
- **Main Loop:** The core of the function iterates through all consecutive scan pairs.
- **ICP Registration:** The `o3d.pipelines.registration.registration_icp` function is called to align the current scan (`source`) with the previous one (`target`). Key aspects include:
    - **Motion Prior (`trans_init`):** The transformation from the previous step is used as an initial guess for the current one. This helps ICP converge faster and avoids getting stuck in local minima.

- **Fitness Check:** The quality of the alignment is checked using the `fitness` score. If it's below a threshold, the registration is considered unreliable, and the algorithm assumes no motion for that step to prevent large errors.

- **Pose and Map Update:** The resulting `current_relative_transform` is composed with the last known global pose to calculate the new global pose. The original, full-resolution scan is then transformed by this new pose and added to the `map_accumulator`.

Stage 3: Saving Results

After the main loop finishes, the estimated trajectory (the list of global poses) must be saved in a standardized format for later analysis and evaluation. The TUM trajectory format (`timestamp tx ty tz qx qy qz qw`) is a common standard for this.

Function:

```
from scipy.spatial.transform import Rotation as R
import numpy as np

def matrix_to_tum_format(timestamp, matrix):

    # Extract translation vector (tx, ty, tz)
    t = matrix[:3, 3]
    # Extract rotation matrix
    rot_matrix = matrix[:3, :3]
    # Convert rotation matrix to a quaternion (qx, qy, qz, qw)
    try:
        q = R.from_matrix(rot_matrix).as_quat()
    except ValueError:
        # Handle cases where the matrix is not a valid rotation (e.g., due to shear)
        q = np.array([0, 0, 0, 1]) # Default to no rotation

    return f"{timestamp} {t[0]} {t[1]} {t[2]} {q[0]} {q[1]} {q[2]} {q[3]}"

def save_trajectory_as_tum(global_poses, all_timestamps, filename):

    print(f"Saving trajectory to {filename}...")
    with open(filename, 'w') as f:
```

```
        for i, pose_matrix in enumerate(global_poses):
            timestamp = all_timestamps[i] if i < len(all_timestamps) else float(i)
            f.write(matrix_to_tum_format(timestamp, pose_matrix) + '\n')
    print(f"Trajectory saved.")
```

Explanation:

- `matrix_to_tum_format`: This utility function takes a 4x4 pose matrix and a timestamp as input. It extracts the translation vector and uses `scipy.spatial.transform.Rotation` to convert the rotation matrix into a quaternion, which is required by the TUM format.

- `save_trajectory_as_tum`: This function iterates through the final list of `global_poses` and uses the helper function to convert each pose into a TUM-formatted string, which is then written to an output text file.

6 Results and Evaluation

After successfully running the ICP-based LiDAR odometry pipeline, the next critical step is to evaluate its performance. This is achieved by comparing the estimated trajectory against a highly accurate ground truth trajectory provided by the dataset. The evaluation provides both quantitative metrics and qualitative insights into the algorithm's accuracy and limitations.

6.1 Evaluation Metrics

To measure the performance of our SLAM front-end, we used two standard metrics for trajectory evaluation:

- **Absolute Trajectory Error (ATE):** This metric measures the global consistency of the entire trajectory. It computes the direct difference between the positions of the estimated trajectory and the ground truth trajectory at each corresponding timestamp. The Root Mean Square Error (RMSE) of these differences gives a single value that represents the overall drift of the system over time. A lower ATE indicates better global accuracy.
- **Relative Trajectory Error (RTE):** This metric evaluates the local accuracy of the odometry. Instead of looking at the global position, it measures the error in the relative motion between consecutive poses. A low RTE suggests that the frame-to-frame scan matching (ICP registration) is performing well, even if small errors accumulate over the long term.

6.2 Quantitative Results

The evaluation script was run on the 2790 poses generated by our ICP algorithm. The results were as follows:

- Absolute Trajectory Error (ATE RMSE): 12.2175 meters
- Relative Trajectory Error (RTE RMSE): 0.0077 meters
- Estimated Accuracy: 94.35% (based on ATE relative to the maximum displacement in the ground truth)

These results are highly informative. The very low RTE of just 0.0077 meters (or 7.7 millimeters) indicates that the point-to-plane ICP registration was extremely accurate on a local, frame-to-frame basis. However, the much larger ATE of 12.2 meters demonstrates the inevitable accumulation of these tiny errors over the full 11.455 km trajectory. This drift is the classic hallmark of an odometry-only system (a SLAM front-end) operating without a back-end for global optimization.

6.3 Qualitative Analysis

The quantitative results are visually confirmed by the plot of the aligned estimated trajectory against the ground truth trajectory.

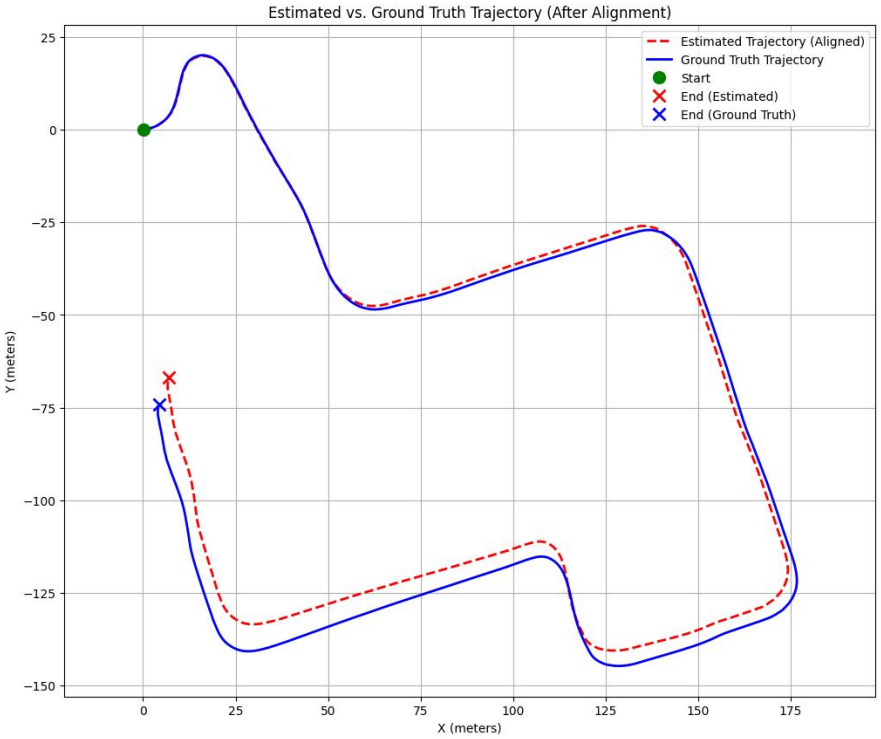


Figure 6: Plot comparing the aligned estimated trajectory (red dashed line) with the ground truth trajectory (blue solid line).

As seen in Figure 6, the overall shape of the estimated path closely matches the ground truth, which is a testament to the accuracy of the ICP front-end. The start points are perfectly aligned. However, as the trajectory progresses, a noticeable drift becomes apparent. The red dashed line gradually deviates from the solid blue line, with the largest error visible at the end of the run. This visual evidence perfectly corresponds with the calculated ATE, showing how small, local inaccuracies compound over time into a significant global error.

The plot clearly illustrates that while the algorithm knows where it is going on a moment-to-moment basis (low RTE), it slowly loses track of its absolute position in the global frame (high ATE).